

TABLE II
COMPARISON BETWEEN VARIOUS PERIPHERALS

PowerMac peripherals	PREP peripherals	IBM system peripherals
UniNorth PCI bridge	PCI Bridge	PCI Bridge
PCI-VGA-compatible graphics card	PCI VGA-compatible graphics card	Virtual I/O network adaptor
Non-volatile RAM	Two IDE interfaces	sPARR non-volatile RAM

Installing QEMU

The method described here built up QEMU from the source. Type the following on the command screen:

1. git clone git://git.qemu-project.org/qemu.git
2. cd qemu
3. git submodule init
4. git submodule update --recursive
5. git submodule status --recursive
6. git checkout stable-2.6
7. mkdir build
8. cd build
9. ./configure --target-list=i386-softmmu instead)
10. make

process allows the guest operating system to think that it is in fact the host operating system of the machine and, hence, has control over its hardware.

Multiple modes of operation

There are four modes in which QEMU operates. Let us take a look at how these help you.

User-mode emulation. In this mode, the emulator runs Linux or Mac OSX programs that were compiled for a different instruction set. Cross-debugging and fast cross-compilation are the main targets in this mode.

System emulation. This mode results in emulation of a full computer system, including peripherals. This mode can be used to provide virtual hosting of several virtual computers on a single computer—that is a lot of virtual for a very real system.

KVM hosting. In order to host a kernel virtual machine (KVM), this mode assists in setting up and migration of KVM images. Though still involved in emulation of hard-



Fig. 2: System Architecture involving QEMU

ware, execution is done as requested by QEMU.

Xen host- ing. In this laziest of modes, all QEMU does is emulate

the hardware, and execution of the guest is totally hidden from QEMU.

QEMU is quick as an emulator, but it works as a virtualiser as well, wherein it can achieve near-native performance by executing the code directly on the host CPU. It is a favourite at EFY because of its extensive reach, be it architectures supported or the interfaces it can handle, and due to it being open source.

There are some minor security flaws associated with QEMU, which are being worked on. As QEMU is written in C, and not Java, it is not easy to port it on to a new system. QEMU also has issues with not being able to utilise multiple host CPUs, rather running all virtual CPUs from a single thread. Unlike similar programs, like VirtualBox and VMware, QEMU does not provide a GUI to manage virtual machines, nor does it provide a way to create persistent virtual machines with saved settings. However, there are several front-ends with GUI for it, like qemu-launcher, qtemu and aqemu.

KVM

A KVM is a Linux kernel module that allows a user program to utilise the hardware virtualisation features of various processors. It supports a host of processor architectures

like Intel, AMD, ARM and MIPS32. QEMU can make use of KVM when it is running a target architecture that is the same as the host architecture. If you are running qemu-system-x64 on an x64-compatible processor, you can utilise HW virtualisation offered by KVM. To use KVM, pass --enable-kvm to QEMU.

How QEMU is different from KVM

KVM is a part of QEMU executable, and different teams are working on both. Where QEMU team focuses on hardware emulation and portability, KVM developers focus on the kernel module and interfacing.

km-qemu executable is pretty qemu-like in its work, that is, it allocates RAM and spawns a new thread. This thread instructs KVM kernel module to switch the system to guest mode before proceeding to execute virtual machine code. KVM does not emulate peripherals of any sort, as it is more of a hypervisor than a virtualiser. Hence, while QEMU can run on a processor without needing hardware virtualisation, KVM needs it.

You still use QEMU, even if you do not

Some of the more famous virtualisation solutions themselves use a layer of QEMU behind all their work.

VirtualBox has a built-in compiler based on QEMU. Xen-HVM, another virtual machine monitor, has device emulation based on QEMU project in order to provide I/O virtualisation to virtual machines.

Another example is KVM, which is a free BSD and Linux kernel module that allows a user space program access to hardware virtualisation. So if you are looking for the best open source solution for all things emulation and virtualisation based, look no further. **EFY**

Ashwin Gopinath, an engineer, who enjoys following innovators, Arduinos and migratory birds